

3RD-PARTY DECLARATIONS • OFFLINE COPY



Complete Build Guide

All three parts in one file. Works offline — copy buttons included. Your progress is saved on this device.

Foundations: the list, the app, and the multi-row form



This part takes you from an empty canvas to a working multi-row declaration form: the **3rd party inventory** list verified, a tablet app connected to it, and a gallery where users add, fill and delete system records — with every keystroke safely stored. **Every step is self-contained**, and every name, number and formula has a copy button.

10 steps

1 list verified

14 fields wired

smoke test



What you're building

THE SERIES IN ONE LINE

A Power Apps declaration form where business users record **one or more third-party applications** (14 required fields each), review them, and submit them all at once into the existing `3rd party inventory` SharePoint list — with an automatic email to the IAM team. Part 1 builds the form itself; Part 2 adds validation and guidance; Part 3 wires up submission, notification and publishing.

THE ARCHITECTURE, IN ONE PARAGRAPH

Every row of the form lives in a local collection called `colSystems` — not in the on-screen controls. Each input *immediately* writes its value back into the collection the moment it changes. That one habit makes the whole app safe: rows can scroll out of sight, rows can be deleted

from the middle, and nothing is ever lost or shuffled, because the collection — not the controls — is the single source of truth. Step A10 will make you prove this to yourself.

TWO PLACEHOLDERS TO SUBSTITUTE

Wherever you see `<YOUR-SITE-URL>` (your SharePoint site) or `<IAM-DL-EMAIL>` (the IAM distribution list address), substitute your own values. Everything else is paste-as-is.

HOW TO READ THIS GUIDE – CLICK VS ENTER

Every step uses the same three markers, so you always know where to look. A navy button like `+ Insert` is something you **click** in Power Apps or SharePoint — a menu, a button, or a property name. An amber chip like `galSystems` is a **value you enter** — tap the chip to copy it (the `📋` mark is the reminder), then paste it where the step says. A key cap like `Ctrl+A` is **pressed on the keyboard**. Dark boxes are complete formulas with their own **Copy** button. Paste, never retype.

HOW EVERY INSERT WORKS

Click `+ Insert` on the left rail → type the control name into the **search box** → click it. The only search terms this guide uses: `Text label` · `Text input` · `Drop down` · `Button` · `Rectangle` · `Blank vertical gallery`. After inserting, rename the control: **double-click its name in the Tree view** → paste the new name → `Enter`.

First, the list

- **A1** Check the 3rd party inventory list

The list already exists — this step makes sure it looks exactly the way the formulas in Part 3 expect. Open <YOUR-SITE-URL> in the browser → **gear icon (top right)** → **Site contents** → click **3rd party inventory**  → **gear icon** again → **List settings**. The **Columns** section lists every column with its **Type** right beside the name — that's where you confirm everything below. Compare it against this table:

#	DISPLAY NAME	EXPECTED TYPE	FORM FILLS IT?
1	Application Name 	Text (likely the renamed Title)	Yes
2	 Vendor/ Provider Name  — space after the /	Text	Yes
3	 Business owner  — lowercase o	Text	Yes
4	Application Purpose 	Multiline text	Yes
5	IT Custodian 	Text	Yes
6	Admin Accounts Used 	Text	Yes
7	 Permissions / Access levels  — lowercase l	Multiline text	Yes
8	Support / Vendor Contact Information 	Text	Yes
9	IAM Role 	Text — assumed	Yes
10	Access Link 	Text	Yes

11	⚠ Division _{SP} — trailing space in the name	Text — assumed	Yes
12	Status 	Text — assumed	Yes
13	Submission Date 	Date and Time — assumed	Yes (auto)
14	IAM Notes 	Text	No
15	Submission Cycle 	Text	No
16	Handover Status 	Text — assumed	Yes
17	Number of Accounts 	Text — assumed	Yes
18	Report Export 	Yes/No	No
19	Onboarded to IAM Platform 	Yes/No	No
20	Access Review Status 	Text	No
21	Review Frequency 	Text	No

⚙ SIX ASSUMED TYPES — GLANCE AND CONFIRM

Six columns marked "assumed" above had ambiguous types; this guide commits to the common-sense ones below (the live list data backs them). While you're in List settings, glance at each — and if one differs, nothing breaks: step C5 (Part 3) has a one-line fix per column, clearly marked.

1. IAM Role  — Single line of text
2. Division_{SP} (the one with the trailing space) — Single line of text
3. Status  — Single line of text
4. Handover Status  — Single line of text
5. Number of Accounts  — Single line of text
6. Submission Date  — Date and Time

Only note a column down if its type is *different* from the list above — that note is all C5 needs.

Two checks and one addition while you're here:

- 1 Title check.** If **Application Name** is the renamed built-in Title column (it usually is), that's fine. What matters: there must be *no separate Title column still marked "Require that this column contains information"* — that would make every submission fail in Part 3. To check: List settings → **Columns** → click **Title** → "Require that this column contains information" must be **No**.
- 2 Add the audit columns** (skip if they already exist). List settings → **Create column** → name **Responder Name** → type **Single line of text** → **OK** (or, from the list view itself: **+ Add column** → **Single line of text**). Repeat for **Responder Email**.

THE QUIRKY NAMES ARE LOAD-BEARING

Four column names contain deliberate-looking accidents —

Vendor/ Provider Name (space after the slash), **Business owner** (lowercase o), **Permissions / Access levels** (lowercase l), and **Division** (trailing space). **Do not rename them** — live data and the formulas in this guide both use them as-is. In Power Apps you'll always accept IntelliSense autocomplete rather than hand-typing these. And when this guide writes **Division_{SP}**, the _{SP} just marks

the invisible trailing space — on screen the column looks like plain "Division".

✓ **CHECK BEFORE MOVING ON**

All 21 columns present with matching names · the six assumed types glanced at (noted any that differ) · no separate required Title column · Responder Name and Responder Email exist as Single line of text columns.

Mark step done

The app shell

◦ **A2** Create the app and set three settings

- 1 Go to make.powerapps.com (check the correct environment, top right) → **Create** → **Start with a blank canvas** → **Tablet size**. If a dialog asks for the app's name right away, paste [Third-Party Application Declaration](#) there. Studio opens with an empty *Screen1*.
- 2 **Settings** (**gear icon** , or **File** → **Settings**) → **Display** : confirm size is [1366](#) × [768](#).
- 3 Settings has a **search box** — type [modern](#) to find *Modern controls and themes* → turn it **OFF**. This guide names classic properties (HintText, Default,

BorderColor) — modern controls would rename half of them.

4 Same search box → type **error**  to find *Formula-level error management* → turn it **ON**. Part 3's submit formula uses IfError, which needs this switch.

5 Tree view → double-click *Screen1* → rename to **scrMain** .

SAVE EARLY

If you weren't asked for a name at creation: **File** → **Save** → paste **Third-Party Application Declaration** . Autosave takes over from here.

✓ CHECK BEFORE MOVING ON

An empty tablet app named and saved · Tree view shows a single screen called **scrMain** · modern controls off · formula-level error management on.

Mark step done

◦ **A3** Connect the SharePoint list

1 Left rail → **Data** (cylinder icon) → **Add data** → search **SharePoint**  → pick the SharePoint connector (sign in if asked).

2 In the site picker, paste <YOUR-SITE-URL> (or pick the

site from Recent) → **Connect**.

- 3 Tick **3rd party inventory** in the list of lists → **Connect**.

NOTHING TO CONFIGURE

The connection brings the columns along, quirky names and all. You won't touch the data source again until Part 3 — but connecting now means IntelliSense already knows every column name when you get there.

✓ CHECK BEFORE MOVING ON

The Data pane lists **3rd party inventory** as a connected data source.

Mark step done

- **A4 App.OnStart — collections, flags, and the first blank row**

Tree view → click **App** → property dropdown (top-left, next to the **fx** bar) → **OnStart** → **Ctrl+A** → paste:

APP · ONSTART

```

Set(gblColorBrand, ColorValue("#C8102E"));
Set(gblColorInk, ColorValue("#1B2436"));
Set(gblColorPaper, ColorValue("#F4F5F8"));
ClearCollect(colIAMRoles, ["Manage", "Govern", "Not
Involved"]);
ClearCollect(colStatusValues, ["Active",
"Decommissioned"]);
ClearCollect(colHandoverValues, ["Handover Not
Required", "Handed Over", "Pending Handover"]);
ClearCollect(colDivisions, ["Operations &
Administration", "Information Technology", "Financial
Planning & Control", "Human Resources", "Strategy &
Transformation", "Marketing & Communications",
"Corporate & Business Development", "Remedial", "In -
Business Risk", "Treasury & Investment", "Private
Banking & Wealth Management", "International Banking",
"Wholesale Banking", "Retail Banking", "Corporate
Secretariat", "Information Security", "Risk
Management", "Credit & Overseas Risk", "Compliance &
AML", "Internal Audit", "Legal"]);
Set(gblShowErrors, false);
Set(gblShowConfirm, false);
Set(gblShowSuccess, false);
Set(gblSubmitting, false);
Set(gblNextSeq, 2);
ClearCollect(colSystems, {RowID: GUID(), SeqNo: 1,
AppName: "", Vendor: "", BizOwner: "", Purpose: "",
Custodian: "", AdminAccts: "", Permissions: "",
SupportContact: "", IAMRole: "", AccessLink: "",
Division: "", Status: "", Handover: "", NumAccts: ""})

```

Then run it once: Tree view → hover  →  →

Run OnStart.

WHAT JUST HAPPENED

Three theme colors · four dropdown source collections (three roles, two statuses, three handover states, **21 divisions**) · four false flags for later parts · a sequence counter starting at 2 · and `colSystems` — the form's single source of truth — seeded with one blank row (SeqNo 1, every field empty).

RUN ONSTART RESETS THE FORM

Re-running OnStart wipes any in-progress rows back to one blank row. While building, that's expected — you'll do it on purpose. Just don't be surprised.

✓ CHECK BEFORE MOVING ON

Left rail → **Variables** (x icon) → Collections: five collections exist. `colSystems` holds exactly 1 row; `colDivisions` holds 21 rows.

Mark step done

◦ **A5** The header band

Four controls on `scrMain` . For each: **+ Insert** → search box → the control's search term → click it → rename in the Tree view (double-click the name → paste → **Enter**) → set X/Y/Width/Height in the **Properties pane** on the right. Two standing motions, used here and in every later step: every quoted **Text** value goes in via property dropdown (top-left, next to the `fx`  bar) → **Text** → **Ctrl+A** → paste (quotes included); every colour goes in via property dropdown → **Color** (or **Fill**) → **Ctrl+A** → paste.

1 + Insert → search **Rectangle** → rename **recHeader** → X **0** · Y **0** · Width **1366** · Height **80**. Property dropdown → **Fill** → **Ctrl+A** → paste: **gblColorBrand**

2 + Insert → search **Text label** → rename **lblAppTitle** → X **24** · Y **10** · Width **800** · Height **36** → Text: **"Third-Party Application Declaration"** → right pane: Font size **22**, **Bold** → Color: **Color.White**.

3 + Insert → search **Text label** → rename **lblAppSubtitle** → X **24** · Y **46** · Width **800** · Height **26** → Text: **"Declare the third-party systems your division uses – all fields are required"** → right pane: Font size **14** → Color: **Color.White**.

4 + Insert → search **Text label** → rename **lblUserBadge** → X **1020** · Y **24** · Width **322** · Height **32** → right pane: Font size **14**, text alignment **Right** → Color: **Color.White**. Then property dropdown → **Text** → **Ctrl+A** → paste:

LBLUSERBADGE · TEXT

```
User().FullName
```

BRAND COLOUR, ONE VARIABLE

`gb1ColorBrand` is the brand red set in OnStart. If branding ever changes, one line in OnStart re-skins the app.

✓ **CHECK BEFORE MOVING ON**

A red band across the top: bold title, smaller subtitle, and your own name sitting right-aligned in the band.

Mark step done

The multi-row form

◦ **A6** The gallery — one card per system

- 1 **+ Insert** → search `Blank vertical gallery` → click it. Rename it `galSystems`.
- 2 The Items panel may pop up asking for a data source — choose `colSystems` (or set the property dropdown → **Items** → **Ctrl+A** → paste `colSystems`).
- 3 Properties pane: X `0` · Y `96` · Width `1366` · Height `580`.
- 4 Property dropdown → **TemplateSize** → **Ctrl+A** → type: `600`
- 5 Now click the **pencil icon** in the gallery's top-left corner — this puts you *inside the template*, so everything you insert next lands in the row, not on the screen. **+ Insert** → search `Rectangle` → rename `recRowCard` → X

8 ▢ · Y 8 ▢ · Width 1318 ▢ · Height 584 ▢. Then three property pastes: **Fill** → **Ctrl+A** → **RGBA(255, 255, 255, 1) ▢** · **BorderColor** → **Ctrl+A** → **RGBA(201, 204, 213, 1) ▢** · **BorderThickness** → **Ctrl+A** → **1 ▢**.

- 6 Still inside the template: + Insert → search **Text** **label ▢** → rename **lblRowNum ▢** → X 16 ▢ · Y 14 ▢ · Width 200 ▢ · Height 28 ▢ → **Bold**. Property dropdown → **Text** → **Ctrl+A** → paste:

LBLROWNUM · TEXT

```
"System " & ThisItem.SeqNo
```

- 1 Still inside the template: + Insert → search **Button ▢** → rename **btnDeleteRow ▢** → X 1180 ▢ · Y 14 ▢ · Width 130 ▢ · Height 32 ▢ → Text: **"Delete Row" ▢** (property dropdown → **Text** → **Ctrl+A** → paste). Property dropdown → **OnSelect** → **Ctrl+A** → paste:

BTNDELETEROW · ONSELECT

```
Remove(colSystems, ThisItem)
```

FIXED HEIGHT, ON PURPOSE

TemplateSize stays a fixed 600. Do not switch to a flexible-height gallery — inputs inside flexible-height galleries misbehave, and this guide's geometry assumes fixed rows.

✓ CHECK BEFORE MOVING ON

One white card in the gallery reading **System 1** with a Delete Row button top-right. (Clicking Delete now removes the only row — Tree view → App → ... → Run OnStart brings it back.)

Mark step done

◦ **A7** Left column — seven fields

Seven label + input pairs, all **inside the gallery template** (pencil icon first — check the breadcrumb above the canvas says you're in `galSystems`). The drill for every field:

+ Insert → search `Text label`, rename, position, set Text (property dropdown → `Text` → `Ctrl+A` → paste, quotes included) · + Insert → search `Text input`, rename, position · property dropdown → `Default` → `Ctrl+A` → paste · property dropdown → `OnChange` → `Ctrl+A` → paste. All labels and inputs: X `16` · Width `630`. Labels Height `20`, inputs Height `40`.

1 · Application Name. Label `lblAppName` at Y `48`, Text `"Application Name"`. Input `txtAppName` at Y `72`:

TXTAPPNAME · DEFAULT

`ThisItem.AppName`

TXTAPPNAME · ONCHANGE

```
Patch(colSystems, ThisItem, {AppName: Self.Text})
```

2 • Vendor/Provider Name. Label `lblVendor` at Y `120`, Text `"Vendor/Provider Name"`. Input `txtVendor` at Y `144`:

TXTVENDOR • DEFAULT

```
ThisItem.Vendor
```

TXTVENDOR • ONCHANGE

```
Patch(colSystems, ThisItem, {Vendor: Self.Text})
```

3 • Business Owner. Label `lblOwner` at Y `192`, Text `"Business Owner"`. Input `txtOwner` at Y `216`:

TXTOWNER • DEFAULT

```
ThisItem.BizOwner
```

TXTOWNER • ONCHANGE

```
Patch(colSystems, ThisItem, {BizOwner: Self.Text})
```

4 • Application Purpose — this one is multiline. Label `lblPurpose` at Y `264`, Text `"Application Purpose"`. Input `txtPurpose` at Y `288`, and additionally property dropdown → **Mode** → `Ctrl+A` → paste `TextMode.MultiLine`:

TXTPURPOSE • DEFAULT


```
ThisItem.Purpose
```

TXTPURPOSE · ONCHANGE

```
Patch(colSystems, ThisItem, {Purpose: Self.Text})
```

5 · IT Custodian. Label `lblCustodian` at Y `336`,
Text `"IT Custodian"`. Input `txtCustodian` at Y
`360`:

TXTCUSTODIAN · DEFAULT

```
ThisItem.Custodian
```

TXTCUSTODIAN · ONCHANGE

```
Patch(colSystems, ThisItem, {Custodian: Self.Text})
```

6 · Admin Accounts Used. Label `lblAdminAccts` at Y
`408`, Text `"Admin Accounts Used"`. Input
`txtAdminAccts` at Y `432`:

TXTADMINACCTS · DEFAULT

```
ThisItem.AdminAccts
```

TXTADMINACCTS · ONCHANGE

```
Patch(colSystems, ThisItem, {AdminAccts: Self.Text})
```

7 · Permissions / Access Levels — multiline again. Label

`lblPermissions` at Y `480`, Text `"Permissions / Access Levels"`. Input `txtPermissions` at Y `504`, Mode → `Ctrl+A` → paste `TextMode.MultiLine`:

TXTPERMISSIONS · DEFAULT

```
ThisItem.Permissions
```

TXTPERMISSIONS · ONCHANGE

```
Patch(colSystems, ThisItem, {Permissions: Self.Text})
```

WHY EVERY INPUT PATCHES ITSELF

`Default` reads the row's stored value; `OnChange` writes edits straight back into `colSystems`. Because both use `ThisItem / Self`, the formulas are identical in every row and every field except the field name — and the collection is always up to date, no matter what the gallery does with its on-screen controls.

✓ CHECK BEFORE MOVING ON

Seven stacked fields down the left half of the card. Type into any of them, then **click somewhere else** — `OnChange` fires when you *leave* the field — and open Variables → `colSystems`: the value is in the collection.

Mark step done

- **A8** Right column — three inputs and four dropdowns

Same drill, right half of the card: everything at X **676** . Width **630** , labels Height **20** , inputs Height **40** . Text values go in the same way as A7 (property dropdown → **Text** → **Ctrl+A** → paste, quotes included). Four of these are **Drop down** controls (+ Insert → search **Drop down**) — set each one's properties in this order: **Items** → **AllowEmptySelection** → **Default** → **OnChange** .

1 • Support / Vendor Contact. Label **lblSupport** at Y **48** , Text **"Support / Vendor Contact"** . Text input **txtSupport** at Y **72** :

TXTSUPPORT • DEFAULT

```
ThisItem.SupportContact
```

TXTSUPPORT • ONCHANGE

```
Patch(colSystems, ThisItem, {SupportContact: Self.Text})
```

2 • IAM Role — first dropdown. Label **lblIAMRole** at Y **120** , Text **"IAM Role"** . **Drop down** **drpIAMRole** at Y **144** . Set **Items** first (box below), then property dropdown → **AllowEmptySelection** → **Ctrl+A** → type **true** , then Default and OnChange:

DRPIAMROLE • ITEMS

```
colIAMRoles
```

DRPIAMROLE · DEFAULT

```
ThisItem.IAMRole
```

DRPIAMROLE · ONCHANGE

```
Patch(colSystems, ThisItem, {IAMRole:  
Self.Selected.Value})
```

3 · Access Link. Label `lblAccessLink` at Y `192`,
Text `"Access Link"`. Text input `txtAccessLink` at
Y `216`:

TXTACCESSLINK · DEFAULT

```
ThisItem.AccessLink
```

TXTACCESSLINK · ONCHANGE

```
Patch(colSystems, ThisItem, {AccessLink: Self.Text})
```

4 · Division. Label `lblDivision` at Y `264`, Text
`"Division"`. **Drop down** `drpDivision` at Y `288`,
AllowEmptySelection `true`:

DRPDIVISION · ITEMS

```
colDivisions
```

DRPDIVISION · DEFAULT

```
ThisItem.Division
```

DRPDIVISION · ONCHANGE

```
Patch(colSystems, ThisItem, {Division:  
Self.Selected.Value})
```

5 • Status. Label `lblStatus` at Y `336`, Text `"Status"`. **Drop down** `drpStatus` at Y `360`, AllowEmptySelection `true`:

DRPSTATUS • ITEMS

```
colStatusValues
```

DRPSTATUS • DEFAULT

```
ThisItem.Status
```

DRPSTATUS • ONCHANGE

```
Patch(colSystems, ThisItem, {Status:  
Self.Selected.Value})
```

6 • Handover Status. Label `lblHandover` at Y `408`, Text `"Handover Status"`. **Drop down** `drpHandover` at Y `432`, AllowEmptySelection `true`:

DRPHANDOVER • ITEMS

```
colHandoverValues
```

DRPHANDOVER • DEFAULT

```
ThisItem.Handover
```

DRPHANDOVER • ONCHANGE

```
Patch(colSystems, ThisItem, {Handover:
Self.Selected.Value})
```

7 • Number of Accounts. Label `lblNumAccts` at Y `480`, Text `"Number of Accounts"`. Text input `txtNumAccts` at Y `504`, and additionally property dropdown → **Format** → `Ctrl+A` → paste `TextFormat.Number`:

TXTNUMACCTS • DEFAULT

```
ThisItem.NumAccts
```

TXTNUMACCTS • ONCHANGE

```
Patch(colSystems, ThisItem, {NumAccts: Self.Text})
```

ALLOWEMPTYSELECTION IS NOT OPTIONAL

A classic dropdown silently auto-selects its first item unless `AllowEmptySelection` is `true` with a blank Default. Skip it and every new row is born with "Manage" / "Operations & Administration" pre-picked — which quietly defeats the mandatory-field validation coming in Part 2.

✓ CHECK BEFORE MOVING ON

Fourteen fields fill the card in two columns. All four dropdowns start **empty**, and each offers the right values (3 roles · 21 divisions · 2 statuses · 3 handover states).

Mark step done

◦ A9 Footer bar — Add System and the row count

Back **out** of the gallery template first — click empty canvas or press `Esc`, and check the breadcrumb: the next three controls belong to `scrMain`, not the template.

1 + Insert → search `Rectangle` → rename `recFooter` → X `0` · Y `692` · Width `1366` · Height `76` → Fill: `gblColorInk` (property dropdown → `Fill` → `Ctrl+A` → paste).

2 + Insert → search `Button` → rename `btnAddSystem` → X `24` · Y `710` · Width `200` · Height `40` → Text: `" + Add System"` (property dropdown → `Text` → `Ctrl+A` → paste). Property dropdown → `OnSelect` → `Ctrl+A` → paste:

BTNADDSYSTEM · ONSELECT

```
Collect(colSystems, {RowID: GUID(), SeqNo: gblNextSeq,
AppName: "", Vendor: "", BizOwner: "", Purpose: "",
Custodian: "", AdminAccts: "", Permissions: "",
SupportContact: "", IAMRole: "", AccessLink: "",
Division: "", Status: "", Handover: "", NumAccts: ""});
Set(gblNextSeq, gblNextSeq + 1)
```

1 + Insert → search `Text label` → rename `lblSystemCount` → X `240` · Y `716` · Width `420` · Height `28` → Color: `Color.White`

(property dropdown → **Color** → **Ctrl+A** → paste). Then property dropdown → **Text** → **Ctrl+A** → paste:

LBLSYSTEMCOUNT · TEXT

```
CountRows(colSystems) & " system(s) in this  
declaration"
```

THE FOOTER ISN'T FINISHED

The Submit button and the form-level error message join this bar in Part 2 — the space to the right is reserved for them.

✓ CHECK BEFORE MOVING ON

Dark footer bar with **+ Add System** and a live count. Click the button: a fresh **System 2** card appears in the gallery and the count says 2. Delete it: the count says 1.

Mark step done

◦ **A10** The smoke test — prove the architecture

Five minutes that buy total confidence in everything Parts 2 and 3 build on. Press **F5** (Preview):

- 1 Click **+ Add System** twice — three cards: System 1, 2, 3.
- 2 Give each row a distinct Application Name — **Alpha** 📄, **Bravo** 📄, **Charlie** 📄 — and fill a couple more fields in each, including a dropdown pick.

- 3 **Scroll test:** scroll the gallery so System 1 leaves the screen, then scroll back. Every value is still there.
- 4 **Delete-the-middle test:** click **Delete Row** on System 2 (*Bravo*). Two cards remain — System 1 still says *Alpha*, System 3 still says *Charlie*. Nothing shifted, nothing bled between rows.
- 5 Leave Preview (**Esc**). Left rail → **Variables** → Collections → **colSystems** : the surviving two rows, exactly as typed. Type one more character in any field and watch the collection update on the next refresh — every keystroke lands in the collection, not just on screen.

WHY WE NEVER READ THE CONTROLS

Galleries recycle their on-screen controls: scroll far enough and the same physical input is redrawn for a different row; delete a row and control state can shift into its neighbour. Apps that read *control values* at submit time lose data to exactly these two behaviours. This app never will — Parts 2 and 3 validate and submit from **colSystems** only, which you have just watched stay correct through both.

✓ CHECK BEFORE MOVING ON

Both tests pass: scrolled-away values persist, and deleting the middle row leaves the other rows' values untouched and correctly paired. The foundation is solid — on to Part 2. 

Mark step done

Guidance, validation, and error messaging



Part 1 left you with a working multi-row form. This part makes it self-explanatory and bullet-proof: placeholder hints on every field, the required-field convention, a submit gate that blocks incomplete declarations, and error messages that name exactly what's missing. **Every name and formula has a copy button** — paste, never retype.

9 steps

10 hint texts

validation gate

error UX

Before you start

PREREQUISITE

Part 1 is complete: your app has the multi-row gallery `galSystems` with all 14 fields writing back into `colSystems`, plus the `btnAddSystem` / `btnDeleteRow` / `lblSystemCount` wiring. If any of that is missing, finish Part 1 first. (Wherever a guide page shows `<YOUR-SITE-URL>` or `<IAM-DL-EMAIL>`, substitute your own values — this part happens to need neither.)

THE ONE RULE FROM PART 1 STILL RUNS EVERYTHING

Validation never looks at input controls — it reads only `colSystems`, which the `OnChange` write-back keeps current on every edit. That's why every formula on this page is a plain expression over the collection: no gallery state, no surprises.

HOW TO READ THIS GUIDE – CLICK VS ENTER

Every step uses the same three markers, so you always know where to look. A navy button like **HintText** is something you **click** in Power Apps – a menu, a button, or a property name. An amber chip like `txtAppName` is a **value you enter** – tap the chip to copy it (the  mark is the reminder), then paste it where the step says. A key cap like `Ctrl+A` is **pressed on the keyboard**. Dark boxes are complete formulas with their own **Copy** button. Paste, never retype.

Guidance – every field explains itself

◦ **B1** Hint text on all ten text inputs

Ten pastes, one per input – the grey guidance a user sees in an empty field. Same motion each time: **Tree view** → expand `galSystems` → click the input → property dropdown (top-left, next to the `fx` bar) → **HintText** → `Ctrl+A` → paste (with the quotes).

TXTAPPNAME • HINTTEXT

"Enter the system/application name"

TXTVENDOR • HINTTEXT

"Enter the software vendor or service provider"

TXTOWNER • HINTTEXT

"Enter the business owner name"

TXTPURPOSE • HINTTEXT

"Describe the business purpose of the application"

TXTCUSTODIAN • HINTTEXT

"Enter the responsible IT contact"

TXTADMINACCTS • HINTTEXT

"List administrator accounts used to manage the system"

TXTPERMISSIONS • HINTTEXT

"Describe available roles and access levels"

TXTSUPPORT • HINTTEXT

"Enter support email, phone number, or contact details"

TXTACCESSLINK • HINTTEXT

"Enter the application URL or access portal link"

TXTNUMACCTS • HINTTEXT

"Enter the total number of active user accounts"

SEEING "PLACEHOLDER" INSTEAD OF HINTTEXT?

You're on a *modern* control — this build uses classic controls throughout (property names like `HintText`, `Default`, `BorderColor`). Revisit Part 1's settings step and make sure "Modern controls and themes" is off, then re-insert the input as a classic one.

✓ CHECK BEFORE MOVING ON

In Preview, a fresh empty row shows grey guidance in all ten text boxes; type one character in any of them and the hint disappears, delete it and the hint returns.

Mark step done

- **B2** Dropdown tooltips — and the empty-until-chosen check

Classic dropdowns have no placeholder, so their guidance rides on **Tooltip**. Four pastes: Tree view → expand **galSystems** → click the dropdown → property dropdown → **Tooltip** → **Ctrl+A** → paste:

DRPIAMROLE • TOOLTIP

"Select IAM involvement level"

DRPDIVISION • TOOLTIP

"Select the applicable division"

DRPSTATUS • TOOLTIP

"Select the application status"

DRPHANDOVER • TOOLTIP

"Select the handover status"

While you're here, re-check **AllowEmptySelection** on all four: select **drpIAMRole** → property dropdown → **AllowEmptySelection** — the formula bar must read **true**. If it reads false, **ctrl+A** → type **true**. Repeat for **drpDivision**, **drpStatus** and **drpHandover**.

WHY ALLOWEMPTYSELECTION MATTERS SO MUCH

Without it, a classic dropdown auto-selects its first item — every new row would arrive with "Manage" pre-chosen, and the mandatory-field validation you're about to build could never catch an untouched dropdown. Empty-until-chosen is what makes "required" real.

✓ CHECK BEFORE MOVING ON

In Preview: hovering each dropdown shows its tooltip, and a freshly added row (**+ Add System**) shows all four dropdowns *blank* — nothing pre-selected.

Mark step done

◦ **B3** Mark every field required

All 14 fields are mandatory, so all 14 labels say so. The first one, done together: Tree view → expand **galSystems** → click **lblAppName** → property dropdown → **Text** → **ctrl+A** → paste:

LBLAPPNAME • TEXT

"Application Name *"

Now the other 13, same motion — each chip is the full Text value (tap to copy, quotes included):

- 1 `lblVendor` → "Vendor/Provider Name *"
- 2 `lblOwner` → "Business Owner *"
- 3 `lblPurpose` → "Application Purpose *"
- 4 `lblCustodian` → "IT Custodian *"
- 5 `lblAdminAccts` → "Admin Accounts Used *"
- 6 `lblPermissions` → "Permissions / Access Levels *"
- 7 `lblSupport` → "Support / Vendor Contact *"
- 8 `lblIAMRole` → "IAM Role *"
- 9 `lblAccessLink` → "Access Link *"
- 10 `lblDivision` → "Division *"
- 11 `lblStatus` → "Status *"
- 12 `lblHandover` → "Handover Status *"
- 13 `lblNumAccts` → "Number of Accounts *"

WANT THE ASTERISK IN BRAND RED?

A classic label is single-colour, so the star inherits the label's ink. That's fine — the red arrives where it counts, on the error borders

and messages you build in B6–B8. Skip the temptation to stack 14 extra one-character labels.

✓ **CHECK BEFORE MOVING ON**

Every field label in the row card ends with * — count 14 in one row, and the layout hasn't wrapped or clipped anywhere.

Mark step done

Validation — the gate and the messages

◦ **B4** The Submit button and the form-level error label

a) The button. Click `scrMain` in the Tree view (so the button lands on the screen, not inside the gallery), then **+ Insert** (left rail) → type `Button` in the search box → click it. Double-click its name in the Tree view and paste `btnSubmit`. Position it via the **Properties pane** on the right → Position → X `1160` · Y `712`, Size → Width `180` · Height `44`. It should sit fully on the dark footer bar next to `btnAddSystem` — if it doesn't, check that `recFooter`'s Y is `692`.

Three properties, each via the property dropdown (top-left, next to the `fx` bar) → select the property → `Ctrl+A` → paste (Text keeps its quotes):

Text

`"Submit Declaration"`

Fill	<code>gblColorBrand</code>
Color	<code>Color.White</code>

Property dropdown → **DisplayMode** → `Ctrl+A` → paste:

BTNSUBMIT · DISPLAYMODE

```
If(CountRows(colSystems) = 0 || gblSubmitting,
DisplayMode.Disabled, DisplayMode.Edit)
```

b) The label. Click `scrMain` in the Tree view again, then **+ Insert** → type `Text label` in the search box → click it. Double-click its name in the Tree view and paste `lblFormError`. Properties pane → Position → X `500` · Y `718`, Size → Width `640` · Height `32`.

Two properties via the property dropdown → select → `Ctrl+A` → paste:

Color	<code>Color.Red</code>
Text	<code>" "</code> — for now; B8 wires the real formula

WHY THE DISPLAYMODE GUARD

An empty declaration (zero rows) has nothing to submit, and `gblSubmitting` will lock the button during the SharePoint write in Part 3 — no double-clicks, no duplicate records.

✓ CHECK BEFORE MOVING ON

With one row in the gallery the button shows solid brand red and is clickable; delete the last remaining row with **Delete Row** → the button greys out; press **+ Add System** and it wakes up again.

Mark step done

◦ **B5** The validation gate on Submit

One paste — the whole gate. Tree view → click

btnSubmit → property dropdown → **OnSelect** →

Ctrl+A → paste:

BTNSUBMIT • ONSELECT

```
Set(gblShowErrors, true);
If(
    CountRows(colSystems) > 0 &&
    CountRows(Filter(colSystems, IsBlank(AppName) ||
    IsBlank(Vendor) || IsBlank(BizOwner) ||
    IsBlank(Purpose) || IsBlank(Custodian) ||
    IsBlank(AdminAccts) || IsBlank(Permissions) ||
    IsBlank(SupportContact) || IsBlank(IAMRole) ||
    IsBlank(Division) || IsBlank(Status) ||
    IsBlank(Handover) || IsBlank(AccessLink) ||
    !IsMatch(AccessLink, "^(https?:/)?([\w\-\]+\.)+[\w\-\]+
    (/S*)?$") || IsBlank(NumAccts) || !IsMatch(NumAccts,
    "^d+$")))) = 0,
    Set(gblShowConfirm, true)
)
```

HOW THE GATE READS

First it switches error display on (**gblShowErrors** — the red borders and messages of B6–B8 key off it). Then it counts invalid rows: a

row fails if *any* of the 14 fields is blank, if the Access Link isn't URL-shaped, or if Number of Accounts isn't a whole number. Only a declaration with **zero** invalid rows sets `gblShowConfirm` — the confirmation popup you build in Part 3.

THE URL PATTERN MUST STAY A LITERAL

`IsMatch` only accepts a pattern typed directly into the formula — never move the regex into a variable "to tidy up"; it will stop compiling. The pattern is deliberately lenient: real inventory entries include bare domains like `stc.com.bh`, and those should pass.

A VALID CLICK DOES NOTHING VISIBLE YET — THAT'S EXPECTED

`gblShowConfirm` flips to true but no popup exists until Part 3. Nothing is broken.

✓ CHECK BEFORE MOVING ON

In Preview with an incomplete row: click **Submit Declaration** → no popup, and the Variables pane (three-dots menu → **Variables**) shows `gblShowErrors = true`, `gblShowConfirm = false`. Fill one row completely — all four dropdowns chosen, Access Link `https://example.com`, Number of Accounts `5` — click again → `gblShowConfirm = true`. Then run OnStart (App → **Run OnStart**) to reset the flags; it re-seeds the gallery to one blank row, which is expected while building.

Mark step done

- **B6** Red borders on all fourteen inputs

Every input signals its own state through **BorderColor**:
resting grey normally, red once **gblShowErrors** is on
and the field is empty (or invalid). The first paste, done
together — Tree view → expand **galSystems** → click
txtAppName → property dropdown → **BorderColor** →
Ctrl+A → paste:

TXTAPPNAME · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.AppName),  
Color.Red, RGBA(201, 204, 213, 1))
```

The other eleven plain fields, same motion — each is a **full paste, no editing**. Select the control → property dropdown
→ **BorderColor** → **Ctrl+A** → paste:

TXTVENDOR · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Vendor),  
Color.Red, RGBA(201, 204, 213, 1))
```

TXTOWNER · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.BizOwner),  
Color.Red, RGBA(201, 204, 213, 1))
```

TXTPURPOSE · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Purpose),  
Color.Red, RGBA(201, 204, 213, 1))
```

TXTCUSTODIAN · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Custodian),  
Color.Red, RGBA(201, 204, 213, 1))
```

TXADMINACCTS · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.AdminAccts),  
Color.Red, RGBA(201, 204, 213, 1))
```

TXTPERMISSIONS · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Permissions),  
Color.Red, RGBA(201, 204, 213, 1))
```

TXTSUPPORT · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.SupportContact),  
Color.Red, RGBA(201, 204, 213, 1))
```

DRPIAMROLE · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.IAMRole),  
Color.Red, RGBA(201, 204, 213, 1))
```

DRPDIVISION · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Division),  
Color.Red, RGBA(201, 204, 213, 1))
```

DRPSTATUS · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Status),  
Color.Red, RGBA(201, 204, 213, 1))
```

DRPHANDOVER · BORDERCOLOR

```
If(gblShowErrors && IsBlank(ThisItem.Handover),  
Color.Red, RGBA(201, 204, 213, 1))
```

The last two carry their format checks as well — full pastes, no editing:

TXACCESSLINK · BORDERCOLOR

```
If(gblShowErrors && (IsBlank(ThisItem.AccessLink) ||  
!IsMatch(ThisItem.AccessLink, "^(https?://)?  
([\w\-\]+\.)+[\w\-\]+(/S*)?$")), Color.Red, RGBA(201,  
204, 213, 1))
```

TXNUMACCTS · BORDERCOLOR

```
If(gblShowErrors && (IsBlank(ThisItem.NumAccts) ||  
!IsMatch(ThisItem.NumAccts, "^\d+$")), Color.Red,  
RGBA(201, 204, 213, 1))
```

WHY BORDERS WAIT FOR A SUBMIT ATTEMPT

Keying everything off `gblShowErrors` means a user filling the form top-to-bottom is never shouted at mid-entry — red appears only after they try to submit, and clears live as they fix each field (the write-back keeps `colSystems` current, and these formulas read `ThisItem`, which *is* the collection row).

✓ CHECK BEFORE MOVING ON

In Preview: click **Submit Declaration** with gaps in a row → red borders on exactly the empty and invalid fields, resting grey on the filled ones. Fill one bordered field and tab out — its border calms down on its own.

- **B7** The per-row error message

Each row card gets one red line at its foot that *names* every offending field — far friendlier than 14 separate error labels, and far fewer controls in the template. Part 1 didn't create this label, so insert it now — **into the gallery template, not onto the screen**:

- 1 Click any control inside the **first** row (e.g. `txtAppName`) so the gallery template is active.
- 2 **+ Insert** (left rail) → type `Text label` in the search box → click it.
- 3 Look at the Tree view: the new label must appear **nested under** `galSystems`. If it sits at screen level instead, delete it, click inside the row again, and re-insert.
- 4 Double-click the label's name in the Tree view → paste `lblRowError` → **Enter**.
- 5 Properties pane (right side) → Position → X `16` · Y `552`, Size → Width `1294` · Height `40`. Leave *Auto height* off (it's off by default) — the gallery template is fixed-height and a growing label would spill out of the card.

6

Property dropdown → Color → Ctrl+A → paste:

Color.Red 

Property dropdown → Text → Ctrl+A → paste:

LBLROWERROR • TEXT

```

If(!gblShowErrors, "",
With({missing:
    Concat(
        Filter(
            Table(
                {f: "Application Name", bad:
IsBlank(ThisItem.AppName)},
                {f: "Vendor/Provider Name", bad:
IsBlank(ThisItem.Vendor)},
                {f: "Business Owner", bad:
IsBlank(ThisItem.BizOwner)},
                {f: "Application Purpose", bad:
IsBlank(ThisItem.Purpose)},
                {f: "IT Custodian", bad:
IsBlank(ThisItem.Custodian)},
                {f: "Admin Accounts Used", bad:
IsBlank(ThisItem.AdminAccts)},
                {f: "Permissions/Access Levels", bad:
IsBlank(ThisItem.Permissions)},
                {f: "Support/Vendor Contact", bad:
IsBlank(ThisItem.SupportContact)},
                {f: "IAM Role", bad:
IsBlank(ThisItem.IAMRole)},
                {f: "Access Link", bad:
IsBlank(ThisItem.AccessLink)},
                {f: "Division", bad:
IsBlank(ThisItem.Division)},
                {f: "Status", bad:
IsBlank(ThisItem.Status)},
                {f: "Handover Status", bad:
IsBlank(ThisItem.Handover)},
                {f: "Number of Accounts", bad:
IsBlank(ThisItem.NumAccts)}
            ), bad), f, ", ")}},
    If(missing <> "", "Missing: " & missing & ". ", ""))

```

```

&
    If(!IsBlank(ThisItem.AccessLink) &&
    !IsMatch(ThisItem.AccessLink, "^((https?:/)?
([\w\-\]+\.)+[\w\-\]+(/S*)?$)",
        "Access Link must be a valid URL (e.g.
https://portal.vendor.com). ", "") &
        If(!IsBlank(ThisItem.NumAccts) &&
        !IsMatch(ThisItem.NumAccts, "^d+$"),
            "Number of Accounts must be a whole number.",
        "")
    ))

```

Then property dropdown → **Visible** → **Ctrl+A** → paste:

LBLROWERROR · VISIBLE

```
gblShowErrors && !IsBlank(Self.Text)
```

HOW ONE LABEL NAMES EVERY GAP

The `Table(...)` builds fourteen tiny {field-name, is-it-bad} records, `Filter` keeps the bad ones, `Concat` strings their names together — then the two trailing `If` s append the URL and whole-number messages when those fields are filled but wrong. One formula, complete diagnosis.

✓ CHECK BEFORE MOVING ON

Submit with a half-filled row → the row's foot reads "Missing: ..." naming exactly the empty fields; put `abc` in Access Link and the URL message joins on. Fix a named field and tab out — it drops off the list immediately.

Mark step done

- **B8** The form-level message, and three finishing touches

a) Wire the footer label from B4. Tree view → click

`lblFormError` → property dropdown → **Text** → **Ctrl+A**

→ paste:

LBLFORMERROR • TEXT

```
If(!gblShowErrors, "",  
With({bad: CountRows(Filter(colSystems,  
IsBlank(AppName) || IsBlank(Vendor) ||  
IsBlank(BizOwner) || IsBlank(Purpose) ||  
IsBlank(Custodian) || IsBlank(AdminAccts) ||  
IsBlank(Permissions) || IsBlank(SupportContact) ||  
IsBlank(IAMRole) || IsBlank(Division) ||  
IsBlank(Status) || IsBlank(Handover) ||  
IsBlank(AccessLink) || !IsMatch(AccessLink,  
"^(https?:/)?([\w\-\-]+\.[\w\-\-]+(/[\S*])?$" ||  
IsBlank(NumAccts) || !IsMatch(NumAccts, "^d+$")))},  
    If(bad > 0, bad & " system record(s) have missing  
or invalid fields – fix the highlighted rows above.",  
    ""))  
))
```

LBLFORMERROR • VISIBLE

```
gblShowErrors && !IsBlank(Self.Text)
```

b) Smoother typing. On each of the ten text inputs: select it → property dropdown → **DelayOutput** → **Ctrl+A** → type **true**. The ten: **txtAppName** · **txtVendor** · **txtOwner** · **txtPurpose** · **txtCustodian** · **txtAdminAccts** · **txtPermissions** · **txtSupport** · **txtAccessLink** · **txtNumAccts**.

Validation now re-evaluates a beat after you pause instead of on every keystroke.

c) Guard Delete Row during submission. Tree view → click **btnDeleteRow** → property dropdown → **DisplayMode** → **Ctrl+A** → paste:

BTNDELETEROW · DISPLAYMODE

```
If(gblSubmitting, DisplayMode.Disabled,  
DisplayMode.Edit)
```

TAB ORDER, FOR FREE

In Preview, **Tab** should walk the left column top-to-bottom, then the right column. Classic controls follow their insertion order — if Part 1 was built in guide order, this already works. If a field is visited out of turn, select it and re-check its position in the Tree view rather than reaching for TabIndex.

✓ CHECK BEFORE MOVING ON

Submit with two incomplete rows → the footer reads "2 system record(s) have missing or invalid fields...". Complete one of them → the count drops to 1 without another click. Complete both → the message disappears entirely.

◦ **B9** The validation test script

A condensed test aimed at everything this page added. Run OnStart first (App → **Run OnStart**) for a clean single blank row, then **F5**:

- **Empty submit** → every field in the row wears a red border, the row foot lists all 14 field names, the footer reads "1 system record(s) have missing or invalid fields...". No popup opens.
- **Bad URL** → fill the row validly but put **not a link** in Access Link → the row message switches to the URL-specific line; only Access Link stays red-bordered.
- **Bad number** → Number of Accounts refuses letters as you type (its Format is Number); paste **12a** anyway → "Number of Accounts must be a whole number." appears. Replace with **25**.
- **Live clearing** → fix the remaining gaps one by one — each border calms and each name drops off the row message as you tab out; the footer count falls to nothing.
- **Valid pass** → click **Submit Declaration** on the fully valid declaration → silence, and the Variables pane shows `gb1ShowConfirm = true`. That silent flag is Part 3's

opening move: the confirmation popup, the SharePoint write, and the IAM email.

HINTS STILL GUIDING

While you're in Preview, glance at a fresh row: grey hint text in every box, tooltips on the dropdowns, an asterisk on every label — the guidance layer and the enforcement layer now tell the same story.

DONE

Validation is complete: nothing incomplete or malformed can reach submission, and every error explains itself. On to Part 3 — confirm, submit, notify, and ship. 🍁

Mark step done

Confirm, submit, notify, and ship

Parts 1 and 2 built a multi-row form that validates itself. This last leg gives the Submit click its consequences: a **confirmation popup**, a batch write to the **3rd party inventory** list with a full audit trail, an **email to the IAM team**, and a success screen that resets for the next declaration. Then you test end-to-end and publish. Every name, number and formula has a copy button.



1 confirmation popup

1 notification flow

1 big submit formula

then publish 🎉

Where you're starting from

PREREQUISITES – PARTS 1 AND 2 ARE DONE

The gallery form adds, edits and deletes rows with every keystroke mirrored into `colSystems`; validation blocks bad submissions with red borders and per-row messages. A fully valid Submit click already sets `gblShowConfirm` — it just has nothing to show yet. That's the first thing we fix.

SUBSTITUTE YOUR VALUES

Wherever you see `<YOUR-SITE-URL>` or `<IAM-DL-EMAIL>`, substitute your own SharePoint site URL and the IAM distribution list address.

HOW TO READ THIS GUIDE – CLICK VS ENTER

Every step uses the same three markers, so you always know where to look. A navy button like **+ Insert** is something you **click** in Power Apps, SharePoint or Power Automate — a menu, a button, or a property name. An amber chip like `btnConfirmYes` is a **value you enter** — tap the chip to copy it (the mark is the reminder), then paste it where the step says. A key cap like `ctrl+A` is **pressed on the keyboard**. Dark boxes are complete formulas with their own **Copy** button. Paste, never retype.

Phase 1 — The confirmation popup

◦ **C1** Build the confirmation popup

Six controls, inserted in this order so they stack correctly (later inserts sit on top). All six get the same **Visible** formula, so the whole popup appears and disappears as one. To rename a control: **double-click its name in the Tree view** → **paste** → `Enter`.

CLICK EMPTY CANVAS BEFORE EACH INSERT

Click an **empty** part of the canvas (or select `scrMain` at the top of the Tree view) before inserting — if a gallery control is selected, the new control lands **inside the gallery** and replicates into every row. Check after each insert: in the Tree view the new control sits at screen level, **not** indented under `galSystems`. If it's nested: delete it, click the screen, insert again.

1 **+ Insert** (left rail) → search `Rectangle` → click it → rename it `recOverlay`. This is the full-screen

dimmer.

- 2 **+ Insert** → search **Rectangle** → rename it **recConfirmCard**. This is the white card.
- 3 **+ Insert** → search **Text label** → rename it **lblConfirmTitle**.
- 4 **+ Insert** → search **Text label** → rename it **lblConfirmMsg**.
- 5 **+ Insert** → search **Button** → rename it **btnConfirmYes** → property dropdown (top-left, next to the fx bar) → **Text** → **ctrl+A** → paste (quotes included): **"Submit"**.
- 6 **+ Insert** → search **Button** → rename it **btnConfirmNo** → property dropdown → **Text** → **ctrl+A** → paste: **"Cancel"**.

Size and place them: select each control, then type these into the **Properties pane (right side)** → **Position** → **X / Y** and **Size** → **Width / Height**. Every number is tap-to-copy:

recOverlay	X 0 · Y 0 · Width 1366 · Height 768
recConfirmCard	X 433 · Y 234 · Width 500 · Height 300
lblConfirmTitle	X 457 · Y 254 · Width 452 · Height

	40
lblConfirmMsg	X 457 · Y 300 · Width 452 · Height 130
btnConfirmYes	X 693 · Y 460 · Width 216 · Height 48
btnConfirmNo	X 457 · Y 460 · Width 216 · Height 48

Fills and fonts **cannot** be pasted into the Properties pane's colour picker — they go through the formula bar. Select `recOverlay` → property dropdown → **Fill** → `Ctrl+A` → paste:

RECOVERLAY · FILL

```
RGBA(0, 0, 0, 0.4)
```

Select `recConfirmCard` → property dropdown → **Fill** → `Ctrl+A` → paste:

RECONFIRMCARD · FILL

```
RGBA(255, 255, 255, 1)
```

Select `lblConfirmTitle` → right pane → Font weight **Bold** · Font size 22.

Now the formulas. Select **each of the six controls in turn** → property dropdown → **Visible** → **Ctrl+A** → paste (a new control's Visible already contains true — **Ctrl+A** replaces it):

ALL SIX POPUP CONTROLS • VISIBLE

```
gblShowConfirm
```

Select **lblConfirmTitle** → property dropdown → **Text** → **Ctrl+A** → paste:

LBLCONFIRMTITLE • TEXT

```
"Confirm submission"
```

Select **lblConfirmMsg** → property dropdown → **Text** → **Ctrl+A** → paste:

LBLCONFIRMMSG • TEXT

```
"You are about to submit " & CountRows(colSystems) & "  
system record(s) to the 3rd party inventory. The IAM  
team will be notified by email. Do you want to  
proceed?"
```

Select **btnConfirmNo** → property dropdown → **OnSelect** → **Ctrl+A** → paste (a new button's OnSelect already contains false):

BTNCONFIRMNO • ONSELECT

```
Set(gblShowConfirm, false)
```

WHY BTNCONFIRMYES STAYS EMPTY FOR NOW

Its OnSelect is the big submit formula — but that formula calls the notification flow, which doesn't exist yet. We build the flow first (C2–C4), then wire the button (C5). The requirement is exactly this popup: a confirmation message with a Submit and a Cancel button.

✓ CHECK BEFORE MOVING ON

In Preview: fill the form validly and press Submit — the screen dims and the card appears with the correct record count in the message. Cancel closes it and the form is untouched. The Submit button on the card does nothing yet — expected.

Mark step done

Phase 2 — The notification flow

◦ C2 Create the flow and its four inputs

The flow is built in Power Automate, in the **same environment** as your app.

- 1 Open <https://make.powerautomate.com> → check the environment picker (top right) matches the app's environment.
- 2 **Create** (left rail) → **Instant cloud flow**.
- 3 Flow name: paste [SystemDeclarationNotify](#).
Choose the trigger **PowerApps (V2)** → **Create**.

4 Click the **PowerApps (V2)** trigger card → **Add an input** → **Text**. The name goes in the box labeled **Input** on the new row — **Ctrl+A** → paste **ResponderName** .

5 **Add an input** → **Text** → **ResponderEmail** .

6 **Add an input** → **Text** → **SubmissionDate** .

7 **Add an input** → **Text** → **SystemCount** .

THE INPUT ORDER IS THE .RUN() SIGNATURE

Create the four inputs in exactly this order — **ResponderName** , **ResponderEmail** , **SubmissionDate** , **SystemCount** . When the app calls **SystemDeclarationNotify.Run(...)** in C5, the arguments are matched *by position*. All four are Text on purpose: the app pre-formats the date and count, which sidesteps timezone and locale conversion entirely.

✓ CHECK BEFORE MOVING ON

The trigger card lists four inputs, top to bottom: **ResponderName**, **ResponderEmail**, **SubmissionDate**, **SystemCount** — all of type Text.

Mark step done

◦ **C3** The email to the IAM team

1 Under the trigger: **New step** (in the new designer: **+ Add an action**) → search **Send an email (V2)** → pick the **Office 365 Outlook** action.

2 **To:** your IAM distribution list — <IAM-DL-EMAIL>.

3 **Subject:** paste exactly:

SEND AN EMAIL (V2) · SUBJECT

New Third-Party Application Submission Received

Body: click the `</>`  (code view) toggle in the body field's top-right, then paste this skeleton:

SEND AN EMAIL (V2) · BODY (HTML)

```
<p>A new third-party application declaration has been submitted.</p>
<ul>
  <li><b>Responder:</b> </li>
  <li><b>Submission date:</b> </li>
  <li><b>Number of declared systems:</b> </li>
</ul>
<p><a href="<YOUR-SITE-URL>/Lists/3rd%20party%20inventory/AllItems.aspx">Open the 3rd party inventory list</a></p>
```

1 Place the cursor after **Responder:** (before the closing `</li>`), then click the lightning-bolt **Dynamic content** button (or the **Add dynamic content** link under the field) → insert the **ResponderName**  chip.

2 Same for **Submission date:** → insert **SubmissionDate** .

3 Same for **Number of declared systems:** → insert

SystemCount .

4 Fix the link: replace <YOUR-SITE-URL> with your real site URL.

5 **Save** the flow.

OPTIONAL: CC THE RESPONDER

Show advanced options (in the new designer: **Advanced parameters**) → CC → insert the `ResponderEmail` dynamic content chip. The submitter gets a copy of what IAM received — a nice audit courtesy, and it's why the app passes the email at all.

NEVER RENAME THIS FLOW LATER

The app will bind to the name `SystemDeclarationNotify`.

Renaming the flow after C4 breaks

`SystemDeclarationNotify.Run(...)` in the app — the fix is to remove and re-add the flow in the Power Automate pane. Name it right once, now, before it ever touches the app.

✓ CHECK BEFORE MOVING ON

The flow shows: PowerApps (V2) trigger with 4 inputs → Send an email (V2) with To = the IAM DL, the exact subject, and a body containing three dynamic-content chips and the list link. Saved without errors.

Mark step done

◦ **C4** **Connect the flow to the app**

- 1 Back in Power Apps Studio (your app open for editing).
- 2 Left rail → **Power Automate** (the flow icon) → **Add flow**.
- 3 Pick **SystemDeclarationNotify** from the list.

FIRST-TIME CONNECTION CONSENT

If a **Connections** screen appears asking to sign in to Office 365 Outlook, click **Sign in / Allow** — the flow can't send mail without it.

✓ CHECK BEFORE MOVING ON

Click into any formula bar and type

`SystemDeclarationNotify.Run(` — IntelliSense autocompletes the flow name and hints four text arguments. Press **Esc** without saving the test-typing.

Mark step done

Phase 3 — Wire the submit

- **C5** The submit formula — write, notify, succeed

One paste. It writes every row of **colSystems** to the list, stamps the audit trail (responder name, email, date/time), calls the flow, and flips to the success screen. It reads **only the collection** — never control state — which is the whole point of the Part 1 architecture.

Tree view → click **btnConfirmYes** → property
dropdown → **OnSelect** → **ctrl+A** → paste (the button's
OnSelect already contains **false** — **ctrl+A** replaces it;
pasting without it breaks the formula):

BTNCONFIRMYES · ONSELECT

```

Set(gblSubmitting, true);
IfError(
    ForAll(colSystems As row,
        Patch('3rd party inventory', Defaults('3rd
party inventory'),
            {
                'Application Name': row.AppName,
                'Vendor/ Provider Name': row.Vendor,
                'Business owner': row.BizOwner,
                'Application Purpose': row.Purpose,
                'IT Custodian': row.Custodian,
                'Admin Accounts Used': row.AdminAccts,
                'Permissions / Access levels':
row.Permissions,
                'Support / Vendor Contact Information':
row.SupportContact,
                'IAM Role': row.IAMRole,
                'Access Link': row.AccessLink,
                'Division ': row.Division,
                Status: row.Status,
                'Handover Status': row.Handover,
                'Number of Accounts': row.NumAccts,
                'Submission Date': Now(),
                'Responder Name': User().FullName,
                'Responder Email': User().Email
            }
        )
    ),
    Set(gblSubmitting, false);
    Notify("Submission failed – please try again. " &
FirstError.Message, NotificationType.Error),
    SystemDeclarationNotify.Run(User().FullName,
User().Email, Text(Now(), "dd mmm yyyy hh:mm"),
Text(CountRows(colSystems)));
    Set(gblShowConfirm, false);

```

```
Set(gblShowErrors, false);  
Set(gblSubmitting, false);  
Set(gblShowSuccess, true)  
)
```

HOW TO PASTE THE QUIRKY COLUMN NAMES

Four column names have traps baked in by the original list:

'Vendor/ Provider Name' (space after the slash), 'Business owner' (lowercase o), 'Permissions / Access levels' (lowercase l), and 'Division ' — with a **trailing space**. The paste above already has them exactly right. If you ever retype one, accept IntelliSense's autocomplete — a hand-typed 'Division' (no space) will not resolve.

IF A COLUMN TYPE DIFFERS FROM THE ASSUMPTION

The formula above is written for the six assumed types from step A1: IAM Role, Division, Status, Handover Status and Number of Accounts as **text**, Submission Date as **Date and Time** — the live list data backs these. If you noted a different type at A1 (or the formula bar flags a line), apply the matching one-line fix:

- Column is **Choice** → wrap in a record: 'IAM Role': {Value: row.IAMRole} — same treatment for 'Division ', Status, 'Handover Status'.
- Number of Accounts is a **Number** column → 'Number of Accounts': Value(row.NumAccts).
- Submission Date is **Text** → 'Submission Date': Text(Now(), "yyyy-mm-dd hh:mm").

Guessed wrong? The formula bar flags that exact line with "expected record value, found text" (or the reverse). Fix just that line per the rules above.

IFERROR NEEDS ITS SWITCH ON

This formula relies on **Formula-level error management**, which you enabled in A2 (**Settings** → search `error`). If `IfError` shows as an unknown function, flip that switch on and re-paste.

✓ **CHECK BEFORE MOVING ON**

No red in the formula bar (after any column-type fixes from the note above). The App checker (stethoscope icon) reports no formula errors on `btnConfirmYes`. Don't test-click yet — the success screen it flips to doesn't exist until C6.

Mark step done

◦ **C6 The success screen and the reset**

Same recipe as C1 — five controls, inserted in order, all sharing one Visible flag. Rename each by double-clicking its name in the Tree view → paste → **Enter**. Geometry goes in the **Properties pane** → **Position / Size**, exactly as in C1:

SAME TRAP AS C1 — CLICK EMPTY CANVAS FIRST

Before every insert, click an **empty** part of the canvas or select `scrMain` in the Tree view; then confirm the new control sits at screen level, **not** indented under `galSystems`. If it's nested: delete it, click the screen, insert again.

- 1 **+ Insert** → search `Rectangle` → rename `recSuccessOverlay` — X `0` · Y `0` · Width `1366` · Height `768`.

2 **+ Insert** → search **Rectangle** → rename
recSuccessCard — X **433** · Y **234** · Width
500 · Height **300**.

3 **+ Insert** → search **Text label** → rename
lblSuccessTitle — X **457** · Y **254** · Width
452 · Height **40** · right pane → Font weight **Bold**
· Font size **22**.

4 **+ Insert** → search **Text label** → rename
lblSuccessMsg — X **457** · Y **300** · Width
452 · Height **130**.

5 **+ Insert** → search **Button** → rename
btnNewDeclaration — X **457** · Y **460** · Width
452 · Height **48** → property dropdown → **Text** →
Ctrl+A → paste: **"Start a new declaration"**.

The two fills go through the formula bar (not the Properties pane). Select **recSuccessOverlay** → property dropdown → **Fill** → **Ctrl+A** → paste:

RECSUCCESSOVERLAY · FILL

```
RGBA(0, 0, 0, 0.4)
```

Select **recSuccessCard** → property dropdown → **Fill**
→ **Ctrl+A** → paste:

RECSUCCESSCARD · FILL

```
RGBA(255, 255, 255, 1)
```

Select **each of the five controls** → property dropdown →

Visible → **Ctrl+A** → paste:

ALL FIVE SUCCESS CONTROLS · VISIBLE

```
gblShowSuccess
```

Select **lblSuccessTitle** → property dropdown →

Text → **Ctrl+A** → paste:

LBLEXUCCESSTITLE · TEXT

```
"Declaration submitted 🎉"
```

Select **lblSuccessMsg** → property dropdown → **Text**

→ **Ctrl+A** → paste:

LBLEXUCCESSMSG · TEXT

```
"Your system records have been added to the 3rd party  
inventory and the IAM team has been notified. You can  
close the app or start a new declaration."
```

Select **btnNewDeclaration** → property dropdown →

OnSelect → **Ctrl+A** → paste:

BTNNNEWDECLARATION · ONSELECT

```
Set(gblNextSeq, 2);
ClearCollect(colSystems, {RowID: GUID(), SeqNo: 1,
AppName: "", Vendor: "", BizOwner: "", Purpose: "",
Custodian: "", AdminAccts: "", Permissions: "",
SupportContact: "", IAMRole: "", AccessLink: "",
Division: "", Status: "", Handover: "", NumAccts: ""});
Set(gblShowSuccess, false)
```

WHY THE RESET MINTS A FRESH GUID

Re-seeding `colSystems` with a brand-new `RowID` forces the gallery to build brand-new items, so no leftover control state from the previous declaration can bleed into the next one. No `Reset()` calls needed — the collection is, as always, the single source of truth.

✓ CHECK BEFORE MOVING ON

Both overlays are invisible in the editor's default state (their flags are false). To eyeball the layout: select `recSuccessOverlay` → property dropdown → **Visible** → **Ctrl+A** → temporarily type `true` — repeat for the card, title, message and button (or Ctrl-click all five in the Tree view and set them once). The card reads cleanly and the button spans under the message. Then restore each Visible to `gblShowSuccess`.

Mark step done

Phase 4 — Prove it, then ship it

- **C7** Check the audit trail with one real record

- 1 Preview the app (**F5**). Fill the single row with obviously-test values (e.g. Application Name **ZZ Test App – delete me** ) , pick any dropdown values, a valid URL and a number.
- 2 Submit → the popup appears → **Submit**.
- 3 The success screen appears. Press **Esc** to leave Preview, then open the **3rd party inventory**  list in the browser.

✓ **CHECK BEFORE MOVING ON**

The newest list item holds your test values in the right columns, and the audit trio is stamped: **Responder Name** is your full name, **Responder Email** is your address, **Submission Date** is just now. SharePoint's built-in **Created By** / **Created** corroborate them. The IAM mailbox (or your CC copy) received the notification. Delete the test item when you're done looking.

Mark step done

◦ **c8 The end-to-end test script**

The full journey, exactly as a business user will drive it. **F5** from the top:

- **+ Add System** twice → three rows. Fill all three completely (distinct app names — Test A, Test B, Test C).

- Submit → popup says **"3 system record(s)"** → press **Cancel** → popup closes, all three rows intact.
- Submit again → **Submit** on the popup → success screen appears.
- In SharePoint: three new items, values in the right columns — spot-check that each row's division landed in the **Division** column (the trailing-space one) and the number in **Number of Accounts**.
- The IAM email arrived: subject exact, responder name right, **Number of declared systems: 3**, and the link opens the list.
- Back in the app: **Start a new declaration** → one blank row, count label says 1, no red anywhere.
- Clean up the three test items in SharePoint.

✓ **CHECK BEFORE MOVING ON**

Every bullet passed — including Cancel preserving the rows and the reset landing on exactly one clean blank row.

Mark step done

○ **c9 The victory lap — publish and share**

1 **File** → **Save** → **Publish** → **Publish this version**.

2 Back on the app's detail page (or File → Share): **Share**

the app with your users or a security group.

- 3 Give the same people **Contribute** permission on the **3rd party inventory**  list — the app writes as the signed-in user, so without list rights their submits fail. The clicks: open the list → **gear icon** → **List settings** → **Permissions for this list** → **Grant Permissions** → enter the users or group → choose **Contribute** — or ask your site owner to do it.

- 4 Flow check: in Power Automate, open **SystemDeclarationNotify**  → **Details** → confirm its connections are healthy and, under **Run only users**, that app users are allowed to run it (adding the flow to the app normally handles this — verify once).

BLUE UNDERLINES ARE FINE

The App checker may show blue delegation hints on formulas that count or filter `colSystems`. Every one of those reads a local collection, not SharePoint — delegation doesn't apply, nothing is truncated, nothing blocks publishing.

✓ DONE

Published, shared, permissions in place, notification flowing. Business users can now declare their third-party systems in batches, IAM gets a mail per submission, and every record carries its responder and timestamp. The build is complete. 🎉

Mark step done

Third-Party Application Declaration — offline build guide. Parts 1–3 combined.
Formulas copy exactly as written; paste, never retype.